

FedKG-Secure :
Secure Retrieval-Augmented Generation Across
Federated Knowledge Graphs

Nidhi Halappanavar

June 2026

Contents

0.1	Overview	2
0.2	Formal Definitions	2
0.2.1	Knowledge Graph	2
0.2.2	Federated Knowledge Graph Setting	3
0.2.3	Query Subgraph Pattern	3
0.2.4	Subgraph Relevance Score	3
0.3	Problem Setting 1: Fixed- k Retrieval with Per-Party Quota	4
0.3.1	Description	4
0.3.2	Simpler Example (Consumer Electronics)	4
0.4	Problem Setting 2: Dynamic- k Retrieval with Secure Global Ranking	5
0.4.1	Description	5
0.4.2	Handling Disjunctive (OR) Semantics	5
0.5	Threat Model and Security Assumptions	6
0.5.1	The Semi-Honest Assumption (Infrastructure Data Owners)	6
0.6	Ideas/ Open Questions	6

Formalisation of Problem Statement

0.1 Overview

We will address the problem of Privacy-Preserving Subgraph Retrieval-Augmented Generation (RAG) over a federation of privately-held, heterogeneous knowledge graphs. Given a natural-language query, the system should retrieve the most semantically and structurally relevant subgraphs from N distributed knowledge graphs that are owned by N mutually distrusting parties without violating the data sovereignty of any party. These retrieved subgraphs will be subsequently used to enhance a Large Language Model (LLM) for answer generation.

0.2 Formal Definitions

0.2.1 Knowledge Graph

A Knowledge Graph in this setting can be defined as a directed, labeled multigraph:

$$G = (V, E, L_V, L_E, \phi_V, \phi_E) \quad (1)$$

Where:

- V - finite set of entity nodes (e.g., `Sony`, `Headphones`, `Bluetooth`)
- $E \subseteq V \times V$ - finite set of directed edges (relations)
- L_V - set of node labels / types (e.g., `Brand`, `Product`, `Technology`)
- L_E - set of edge labels / relation types (e.g., `manufactures`, `uses_protocol`)
- $\phi_V : V \rightarrow L_V$ and $\phi_E : E \rightarrow L_E$ are labeling functions mapping nodes and edges to their respective types.

Nodes and edges carry attribute vectors (embeddings) for semantic space operations.

Motivation for this definition: The explicit formalization of labels, mapping functions, and continuous embeddings solves the core conflict between shared structural logic and local data sovereignty in a federation.

- **Privacy through Abstraction (ϕ_V, ϕ_E):** Parties cannot safely share their internal database schemas or primary keys. The mapping functions thereby will allow each party to project their private, local data (V, E) onto a shared, high-level taxonomy (L_V, L_E). This will allow the orchestrator to verify the topological “shape” of the data (e.g., recognizing that a `Brand` node points to a `Product` node) without ever exposing the underlying private records.
- **Bridging Ontological Drift (Embeddings):** In a rigid database, a query for `Smartphone` will fail if Party B uses the label `Mobile_Phone`. By attaching continuous semantic embeddings to nodes and edges, the system would not rely on exact string matches. It instead will measure the vector distance, instantly recognizing that `Mobile_Phone` and `Smartphone` are mathematically synonymous.

0.2.2 Federated Knowledge Graph Setting

Let there be N parties, each owning a private knowledge graph(KG), forming the federation:

$$\mathcal{F} = \{G_1, G_2, \dots, G_N\} \quad (2)$$

Where G_i is the private KG of party i . The graphs share a common ontological schema (L_V, L_E) , but their instance data (V_i, E_i) is strictly private.

Constraints:

1. The intersection $G_i \cap G_j$ is strictly unknown to the parties.
2. No trusted third party has plaintext access to the union graph $\bigcup_{i=1}^N G_i$.
3. All cross-party computations will be mediated by a secure protocol.

0.2.3 Query Subgraph Pattern

A natural-language query q will be mapped by an LLM into a structured query subgraph pattern:

$$Q = (V_Q, E_Q, c_V, c_E) \quad (3)$$

Where:

- V_Q - query anchor nodes (e.g., {Sony, Headphones, x } where x is a variable).
- E_Q - query edges (e.g., {(Sony, manufactures, x)}, (x , is_a, Headphones)}).
- c_V and c_E - functions mapping query nodes and edges to semantic embedding vectors.

0.2.4 Subgraph Relevance Score

For a candidate subgraph $S_i \subseteq G_i$, relevance will be measured by jointly enhancing structural alignment and semantic proximity (analogous to SimGRAG):

$$\text{Relevance}(S_i, Q) = \alpha \cdot \text{ISO}(S_i, Q) + \beta \cdot \text{SemDist}(S_i, Q) \quad (4)$$

Where ISO denotes structural isomorphism (via Graph Edit Distance or soft matching), SemDist is the semantic embedding distance, and α, β are weighting hyperparameters.

The Necessity of Soft Structural Matching for Disjunctive Federation:

In a multi-party federation, heterogeneous Knowledge Graphs will undoubtedly exhibit topological differences. The same semantic concept may be represented as a direct 1-hop edge in Party 1’s KG, but as a multi-hop path in Party 2’s KG.

When executing disjunctive (OR) queries, the LLM will generate a unified query pattern Q that attempts to capture multiple potential semantic branches. If the system were to enforce *strict* structural isomorphism (where candidate subgraphs must be bijectively identical to Q), it would impose a rigid, binary filter that fails to accommodate the localized schema differences between Party 1 and Party 2. Consequently, valid subgraphs fulfilling the OR condition would be discarded due to minor topological mismatch.

By defining structural alignment as a continuous function $\text{ISO}(S_i, Q)$ utilizing Graph Edit Distance (GED), the system will become topology-tolerant. This soft approach ensures that subgraph candidates from diverse parties are not penalised for beneficial schema heterogeneity, allowing the joint relevance function to gracefully rank diverse OR branches across the federation based on integrated structural resemblance and semantic proximity.

0.3 Problem Setting 1: Fixed- k Retrieval with Per-Party Quota

0.3.1 Description

The retrieval budget is uniformly pre-allocated. Each party i independently retrieves exactly k_{local} subgraphs from its local KG. The orchestrator then securely ranks the $N \times k_{\text{local}}$ candidates to find the global top- k .

$$\text{Step 1: } \forall i \in \{1 \dots N\} : \mathcal{C}_i = \text{TopK}_{\text{local}}(G_i, Q, k_{\text{local}}) \quad (5)$$

$$\text{Step 2: } \mathcal{R} = \text{TopK}_{\text{global}} \left(\bigcup_{i=1}^N \mathcal{C}_i, k_{\text{global}} \right) \quad (6)$$

0.3.2 Simpler Example (Consumer Electronics)

To understand the inherent limitations of a fixed-quota retrieval system, we must examine the hidden distribution of relevance scores across a skewed dataset.

Scenario Setup:

- **Query:** “Which smartphones use OLED displays?”
- **Query Graph Q :** (Smartphone) -[uses_component]-> (OLED_Display)
- **Target Retrieval:** A global target of $k_{\text{global}} = 4$ subgraphs.
- **The Constraint:** A uniform fixed quota where $k_{\text{local}} = 2$ per party.
- **The Federation ($N = 3$):** Party A (Premium Mobile Vendor with deep OLED data), Party B (Budget Mobile Vendor with limited OLED data), and Party C (Audio Equipment Vendor with no phone data).

The Underlying Data (Hidden Local Scores):

If we evaluate the absolute best internal matches within each party’s silo, the true relevance scores are distributed as follows:

Party	Candidate Subgraph (S)	Relevance Score
Party A	S_{A1} : iPhone 15 Pro OLED specs	0.98
Party A	S_{A2} : Galaxy S24 Ultra OLED specs	0.96
Party A	S_{A3} : Pixel 8 Pro OLED specs	0.95
Party A	S_{A4} : OnePlus 12 OLED specs	0.92
Party B	S_{B1} : Older Galaxy A54 OLED	0.75
Party B	S_{B2} : Budget Moto G OLED	0.70
Party C	S_{C1} : Headphone LED charging light	0.20
Party C	S_{C2} : Speaker Bluetooth display	0.15

Step 1: Local Quota Execution

Because the system enforces $k_{\text{local}} = 2$, each party independently submits only its top 2 subgraphs to the secure aggregator.

- **Party A** submits S_{A1} (0.98) and S_{A2} (0.96). It is forced to withhold S_{A3} (0.95) and S_{A4} (0.92).

- **Party B** submits S_{B1} (0.75) and S_{B2} (0.70).
- **Party C** submits S_{C1} (0.20) and S_{C2} (0.15).

Step 2: Global Aggregation and Limitation

The aggregator compiles a pool of $3 \times 2 = 6$ subgraphs and selects the global top $k = 4$. The final retrieved set is $\{0.98, 0.96, 0.75, 0.70\}$.

The Limitation: The true mathematical top-4 across the entire federation actually resided entirely within Party A ($\{0.98, 0.96, 0.95, 0.92\}$). The fixed-quota system actively suppressed highly relevant knowledge (scores 0.95 and 0.92) to accommodate the quota of a less relevant party, thereby degrading the factual grounding provided to the LLM.

0.4 Problem Setting 2: Dynamic- k Retrieval with Secure Global Ranking

0.4.1 Description

The retrieval budget k is evaluated globally across the secure federation. Allocation is purely merit-based; party contributions $|\mathcal{R}^* \cap G_i|$ dynamically range from 0 to k .

$$\mathcal{R}^* = \underset{|\mathcal{R}|=k}{\mathcal{R} \subseteq \bigcup_{i=1}^N G_i} \sum_{S \in \mathcal{R}} \text{Relevance}(S, Q) \quad (7)$$

0.4.2 Handling Disjunctive (OR) Semantics

Real-world queries often require disjunctive matching, where the system must resolve complex boolean logic across heavily skewed data distributions.

Scenario Setup:

- **Query:** “Compare the active noise-canceling (ANC) technology of Sony OR Bose premium headphones.”
- **Query Graph Q :** (Sony OR Bose) -[has_feature]-> (Active_Noise_Cancellation)
- **Target Retrieval:** $k = 5$ subgraphs.
- **The Federation ($N = 3$):** Party 1 (Sony Distributor), Party 2 (Bose Distributor), and Party 3 (PC Parts Vendor with no headphone data).

The Underlying Data (Hidden Local Scores):

If we evaluate the absolute best internal matches within each party’s silo system, the true relevance scores are distributed as follows:

Party	Candidate Subgraph (S)	Relevance Score
Party 1	S_{1a} : Sony WH-1000XM5 ANC processor	0.95
Party 1	S_{1b} : Sony dual-noise sensor tech	0.92
Party 1	S_{1c} : Sony adaptive sound control	0.89
Party 2	S_{2a} : Bose QuietComfort Ultra ANC	0.94
Party 2	S_{2b} : Bose CustomTune audio calibration	0.90
Party 3	S_{3a} : PC case cooling fan noise	0.15

The Failure of Fixed Quotas (Setting 1):

If the system enforces a strict local quota (e.g., $k_{\text{local}} = 2$), Party 1 is artificially capped and must withhold S_{1c} (0.89). Conversely, Party 3 is forced to contribute its highest-scoring, yet entirely irrelevant, subgraph S_{3a} (0.15). The final aggregated top-5 set would erroneously include the PC cooling fan data while discarding highly relevant Sony data.

The Dynamic Resolution (Setting 2):

Under Dynamic- k retrieval, the secure protocol evaluates the relevance bounds globally. The system successfully identifies the absolute mathematical top 5 scores across the entire network: $\{0.95, 0.94, 0.92, 0.90, 0.89\}$.

The retrieval allocation perfectly mirrors the actual distribution of knowledge:

- **Party 1 contributes 3 subgraphs** (S_{1a}, S_{1b}, S_{1c})
- **Party 2 contributes 2 subgraphs** (S_{2a}, S_{2b})
- **Party 3 contributes 0 subgraphs**

The final $k = 5$ set accurately reflects the best available data across the entire network, gracefully handling the OR logic without arbitrary local caps or polluting the LLM context window with irrelevant data.

0.5 Threat Model and Security Assumptions

0.5.1 The Semi-Honest Assumption (Infrastructure Data Owners)

The project will assume an Honest-but-Curious (Semi-Honest) adversarial model for the federation parties (P_{data}) and the orchestrating computation shards (P_{compute}).

- **Honest Execution** : The computation servers will honestly execute the SMPC protocols according to the design specifications. They will neither drop arbitrary messages nor provide malicious inputs.
- **Curious Observers** : Under this model, servers will follow the protocol strictly, but they may attempt to learn as much information as possible from the secret shares, intermediate outputs, and network communications observed during execution.
- **Non-Collusion** : We will assume a non-collusion among the majority of the computation servers within any given shard. No majority coalition of servers will collaborate to reconstruct the plaintext data from the secret shares.

0.6 Ideas/ Open Questions

To fully actualize this dynamic, multi-party retrieval framework, several optimization and privacy enhancements remain open for investigation. Below are the core architectural challenges, broken down into their underlying mechanics and the resulting open questions:

- **Mitigating Communication Overhead via Fixed-Shard Topologies**

- *The Context:* As the federation grows, performing monolithic global nMPC would introduce a severe network bandwidth bottlenecks. We could simplify this by transitioning to a *fixed-shard communication topology*, dividing the federation into smaller, static computational groups. Each shard will securely compute an intermediate top- k list and forwards only this reduced subset to a global aggregator, drastically cutting cross-network traffic. If the global aggregator finds the first submission of a shard to be insufficient, it can send a secure follow-up request to that shard for more candidates without restarting the entire protocol, similar to recomputation in pipeline parallelism. Importantly, this request should not reveal the current global cut-off score to the shard, as that would expose the shard’s relative position in the federation.
- **Open Question 1: How can we design fixed-shard networks that prevent the "dropping problem" – where shard-level pre-filtering prematurely discards globally competitive subgraphs before the final merge?**
- **Open Question 2: Static and public shard membership is an inherent tension with privacy: an adversary who knows which parties share a shard can craft targeted queries to infer the composition of that shard over time. Is this leakage acceptable under a realistic threat model, or does it require shard membership itself to be kept private – and if so, at what cost?**

- **Tracking the Dynamic- k Boundary via Oblivious Data Structures**

- *The Context:* Re-sorting the entire union of global candidates from scratch via traditional Secure Sorting Networks for every query would be computationally prohibitive. Alternatively, maintaining a running top- k boundary using a Secure Priority Queue with lightweight cryptographic hash commitments would allow the system to conceal subgraph content until selection is confirmed. This makes full Oblivious RAM (ORAM) unnecessary for this setting, since our threat model requires hiding subgraph *content* rather than *access patterns*—the problem ORAM was designed to solve.
- **The Open Question: Every oblivious insertion into a priority queue requires expensive cryptographic operations (e.g., Oblivious Transfer or garbled gates). Can the constant factors of these operations be optimised enough to make this approach empirically faster than global sorting at realistic federation sizes, or does the per-operation cryptographic cost dominate any asymptotic advantage?**

- **Defending Vector Embeddings via Differential Privacy (DP)**

- *The Context:* SMPC successfully hides *who* contributed data during computation, but revealing the final semantic distance (SemDist) scores creates a residual vulnerability. Through repeated queries, an adversary could track these scores to reverse-engineer the underlying graph embeddings via membership inference attacks. We could defend against this by applying central

output perturbation – injecting a calibrated DP noise into the final scores just before results are revealed to the LLM.

- *The Open Question:* **Where is the optimal operating point on the privacy-utility trade-off curve – specifically, how much DP noise can be injected before the perturbed rankings diverge enough from the true rankings to meaningfully degrade the factual grounding of the LLM’s generated answers?**

- **Possibility of Offline pre-processing?**

- *The Context:* In order to mitigate the severe online latency of global SMPC, the federation could shift to an offline pre-processing model, where data can be partitioned, clustered, or indexed before any query arrives. However, just as static shard membership creates an inherent tension with privacy, any form of offline partitioning could introduce a critical vulnerability regarding access patterns. If the mapping of data to these pre-computed partitions remains static, the system’s behaviour can become predictable.
- *The Open Question:* **The Open Question: If the system relies on offline partitioning to accelerate retrieval, how can we prevent an adversary from exploiting access pattern leakage? By systematically observing which static partitions are repeatedly activated (or updated) by specific types of queries over time, an adversary could reliably infer the underlying semantic distribution of the federation’s private data.**

Algorithm 1 Federated Semantic-Structural Retrieval

Require: Query graph $Q = (V_Q, E_Q)$, parties $\mathcal{P} = \{P_1, \dots, P_m\}$, top- K , mode μ

Ensure: Top- K matched paths with scores and party traces

```
1:  $V_{\text{known}} \leftarrow \{v \in V_Q \mid v \text{ is not UNKNOWN}\}$ 
2:  $V_{\text{unknown}} \leftarrow \{v \in V_Q \mid v \text{ is UNKNOWN}\}$ 
3:  $R_Q \leftarrow \{r \mid (u, r, v) \in E_Q, r \text{ is not UNKNOWN}\}$ 
4: for each party  $P_i \in \mathcal{P}$  in parallel do
5:    $T_i \leftarrow V_{\text{known}} \cup R_Q$ 
6:    $Z_i \leftarrow \text{Emb}(T_i)$ 
7:   Search  $P_i$ 's node index using  $Z_i[0 : |V_{\text{known}}|]$ 
8:   Search  $P_i$ 's relation index using  $Z_i[|V_{\text{known}}| :]$ 
9:   Obtain local node candidates  $C_i^V$  and relation candidates  $C_i^R$ 
10:  for each  $u \in V_{\text{unknown}}$  do
11:    if  $P_i$  has no type constraints then
12:       $C_i^V(u) \leftarrow \text{NONE}$ 
13:    else if a matching local type exists for  $u$  then
14:       $C_i^V(u) \leftarrow \{(x, 0) \mid x \text{ has that type in } P_i\}$ 
15:    else
16:       $C_i^V(u) \leftarrow \emptyset$ 
17:    end if
18:  end for
19: end for
20:  $C^V(q) \leftarrow \bigcup_{i=1}^m C_i^V(q)$  for every query node  $q$ 
21:  $C^R(r) \leftarrow \bigcup_{i=1}^m C_i^R(r)$  for every query relation  $r$ 
22:  $v_{\text{root}} \leftarrow \arg \min_{v: C^V(v) \neq \emptyset} |C^V(v)|$ 
23:  $S \leftarrow \text{DFSOrder}(Q, v_{\text{root}})$ 
24: Compute greedy lower bounds  $LB$  over  $S$ 
25: Initialize top- $K$  result set  $\mathcal{R}$ 
26: for each  $x_{\text{root}} \in C^V(v_{\text{root}})$  sorted by score do
27:    $M[v_{\text{root}}] \leftarrow x_{\text{root}}$ 
28:    $A[v_{\text{root}}] \leftarrow \text{AlignKey}(x_{\text{root}})$ 
29:    $\text{Match}(1, M, A, [], d(v_{\text{root}}, x_{\text{root}}), \text{False}, [])$ 
30: end for
31: if  $\mu = \text{HYBRID}$  and greedy results are not confident then
32:   for  $B \in \{16, 32, 64\}$  do
33:     Rerun Match using adaptive beam width  $B$ 
34:     Merge beam results into  $\mathcal{R}$ 
35:   end for
36: end if
37: return  $\mathcal{R}$ 
```

Algorithm 2 Federated Structural Matching Procedure

```
1: procedure MATCH( $j, M, A, E, s, b, \Pi$ )
2:   if  $j > |S|$  then
3:     Add  $(s, E, b, \Pi)$  to  $\mathcal{R}$ 
4:     return
5:   end if
6:    $(u, r, v) \leftarrow S[j]$ 
7:    $x \leftarrow M[u]$ 
8:    $\mathcal{X} \leftarrow []$ 
9:   for each alias  $x'$  where  $\text{AlignKey}(x') = \text{AlignKey}(x)$  do
10:    Let  $P_i$  be the party owning  $x'$ 
11:     $R_{\text{allow}} \leftarrow \{\rho.\text{id} \mid \rho \in C^R(r), \rho.\text{party} = P_i\}$ 
12:    if  $C^V(v) = \text{NONE}$  then
13:       $V_{\text{allow}} \leftarrow \text{NONE}$ 
14:    else
15:       $V_{\text{allow}} \leftarrow \{y.\text{id} \mid y \in C^V(v), y.\text{party} = P_i\}$ 
16:    end if
17:     $\mathcal{X} \leftarrow \mathcal{X} \cup \text{Expand}_{P_i}(x', R_{\text{allow}}, V_{\text{allow}})$ 
18:  end for
19:  Remove duplicate expansions from  $\mathcal{X}$ 
20:   $\mathcal{C} \leftarrow []$ 
21:  for each expansion  $e = (x', \rho, y) \in \mathcal{X}$  do
22:     $k_y \leftarrow \text{AlignKey}(y)$ 
23:    if  $v \in A$  and  $A[v] \neq k_y$  then
24:      continue
25:    end if
26:    if  $v \in A$  or  $C^V(v) = \text{NONE}$  then
27:      nodeScore  $\leftarrow 0$ 
28:    else
29:      nodeScore  $\leftarrow d(v, y)$ 
30:    end if
31:    relationScore  $\leftarrow d(r, \rho)$ 
32:     $s' \leftarrow s + \text{nodeScore} + \text{relationScore}$ 
33:    Add  $(s', e)$  to  $\mathcal{C}$ 
34:  end for
35:  Sort  $\mathcal{C}$  by increasing  $s'$ 
36:  if  $\mu = \text{ADAPTIVE\_BEAM}$  then
37:    Apply diversity beam slicing to  $\mathcal{C}$ 
38:  end if
39:  for each  $(s', e = (x', \rho, y)) \in \mathcal{C}$  do
40:    if  $\mu = \text{GREEDY}$  and  $s' + LB[j + 1] > \text{WorstScore}(\mathcal{R})$  then
41:      break
42:    end if
43:     $M' \leftarrow M \cup \{v \mapsto y\}$ 
44:     $A' \leftarrow A \cup \{v \mapsto \text{AlignKey}(y)\}$ 
45:     $E' \leftarrow E \cup [(x', \rho, y)]$ 
46:     $\Pi' \leftarrow \Pi \cup [\text{party}(x')]$ 
47:    Match( $j + 1, M', A', E', s', b, \Pi'$ )
48:  end for
49: end procedure
```

Algorithm 3 Inline Rich RAG Evidence Collection

Require: Scored same-step expansions $\mathcal{C}$, selected answer edge e^* , query relation r , limit L

Ensure: Supporting evidence edges H_j

```
1: procedure SAMESTEPSUPPORT( $\mathcal{C}, e^*, r, L$ )
2:    $H_j \leftarrow []$ 
3:    $\sigma^* \leftarrow (\text{AlignKey}(e^*.source), e^*.relation, \text{AlignKey}(e^*.target))$ 
4:   for each expansion  $e = (x, \rho, y)$  in  $\mathcal{C}$  do
5:      $\sigma \leftarrow (\text{AlignKey}(x), \rho, \text{AlignKey}(y))$ 
6:     if  $\sigma = \sigma^*$  then
7:       continue
8:     end if
9:     if  $\rho.id = r$  then
10:      reason  $\leftarrow$  "same-step alternative"
11:    else
12:      reason  $\leftarrow$  "semantic same-step alternative"
13:    end if
14:    Add  $(x, \rho, y, \text{party}(x), \text{reason})$  to  $H_j$ 
15:    if  $|H_j| \geq L$  then
16:      break
17:    end if
18:  end for
19:  return  $H_j$ 
20: end procedure
21: During each matching step, after selecting  $e^*$ :
22:  $H_j \leftarrow \text{SameStepSupport}(\mathcal{C}, e^*, r, L)$ 
23: Pass  $H \cup H_j$  to the next recursive call
24: At completion, deduplicate  $H$  and keep at most  $T$  edges
```

